

自动化测试体系优化方案

一、自动化问题背景

当前自动化测试问题：

- 失败用例无法快速定位根因
- 用例间状态依赖问题难以复现
- 隐性问题（顺序依赖 / 长时间运行）发现能力不足
- 自动化结果未形成长期可复用资产

因此，有必要在现有 **客户端自动化测试工具** 与 **Jenkins 自动化平台** 基础上，构建一套更智能、更可分析、更可回归的测试体系。

二、设计目标

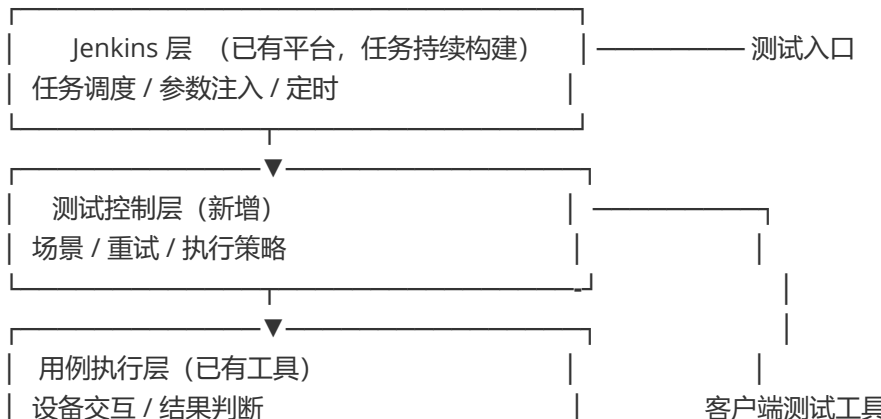
- 用例失败 **可复现**
- 失败原因 **可分析**
- 相似失败 **可聚类**
- 失败场景 **可回归**

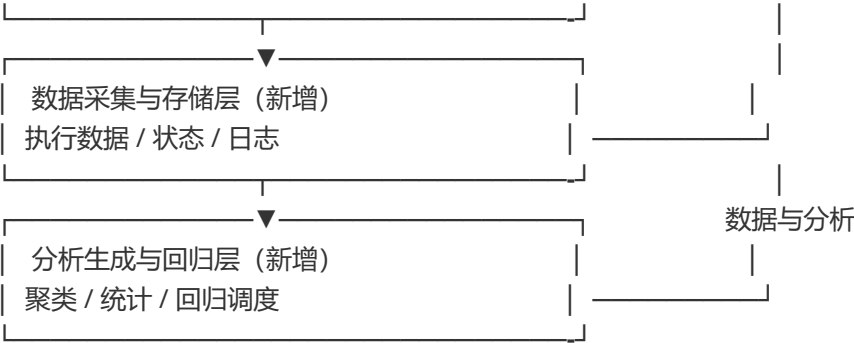
三、整体架构说明

3.1 系统组成

组件	职责
Jenkins	任务调度、参数注入、定时触发
客户端测试工具	用例执行、设备交互、状态采集
测试数据与分析中心	失败用例库、失败场景库、统计分析

3.2 三大组件对应下列不同应用层





1.Jenkins层

- 要触发任务，对应不同项目完成日常任务构建

2.测试控制层

- 控制用例执行顺序
- 控制失败重试
- 控制场景执行
- 管理用例生命周期

3.用例执行层

- 客户端工具

4.数据采集与存储层

层级	示例
系统层	心跳 / MEM / 重启信息
连接层	网络 / 蓝牙
业务层	模式 / 音量 / 链路耗时
算法层	asr / 门限 / 识别耗时

其中数据采集还可以同步当前状态变化数据到客户端，及时保存各状态设备信息

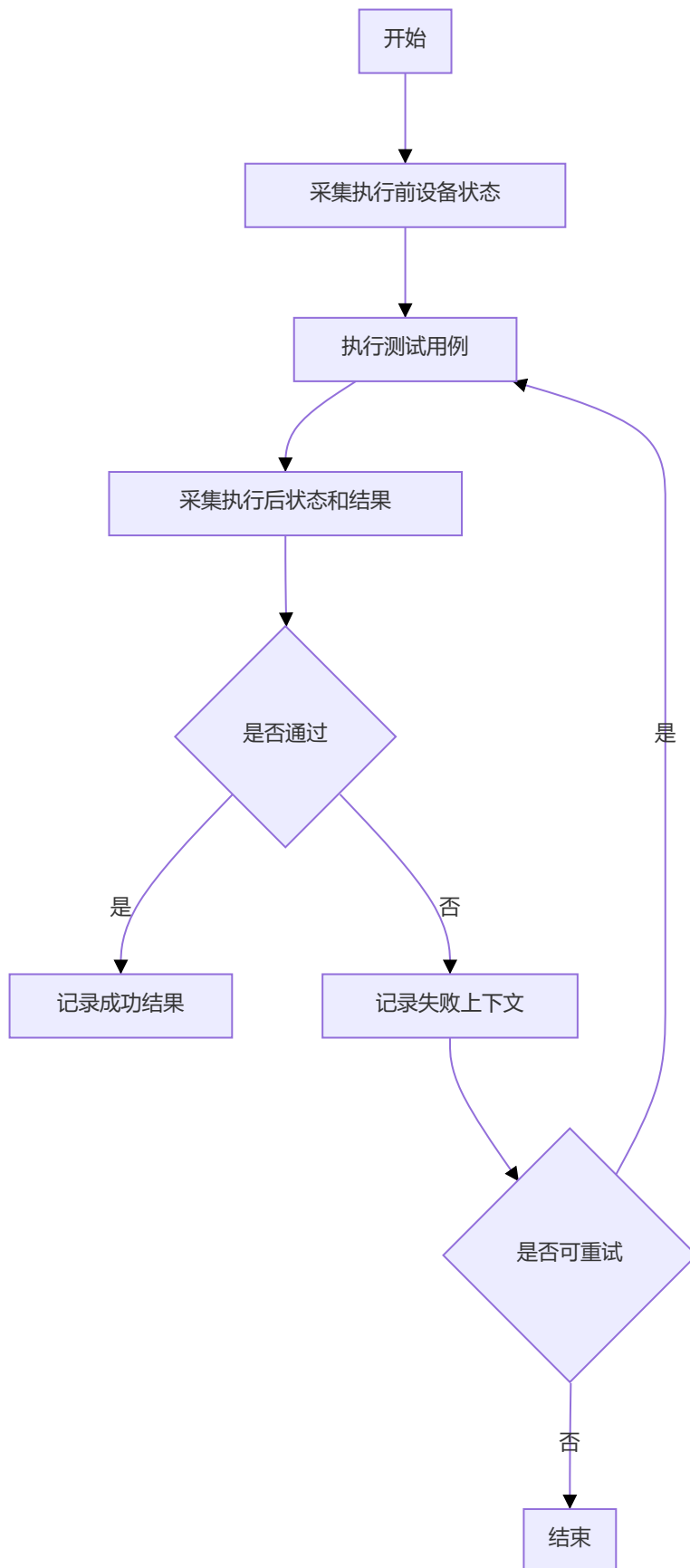
5.分析生成与回归层

- 场景定义
 - 后续主要的工作内容之一，根据不同验证需求定义指定环境下的不同测试场景
- 场景生成及生成策略定制
 - 普通随机生成
 - 指定策略扰动：wakeup -- asr -- asr --- network_disconnctet --asr
 - 权重生成
- 失败用例、场景分析
 - 单用例|单场景分析（对功能验证）
 - 结合上下文分析（对失败场景发掘）
 - 高危用例及场景标记

其中场景生成策略、失败用例|场景定义策略、高危用例标记策略为可变模块或编辑策略池，不植入工具，当作模块选择。

四、核心流程与数据说明

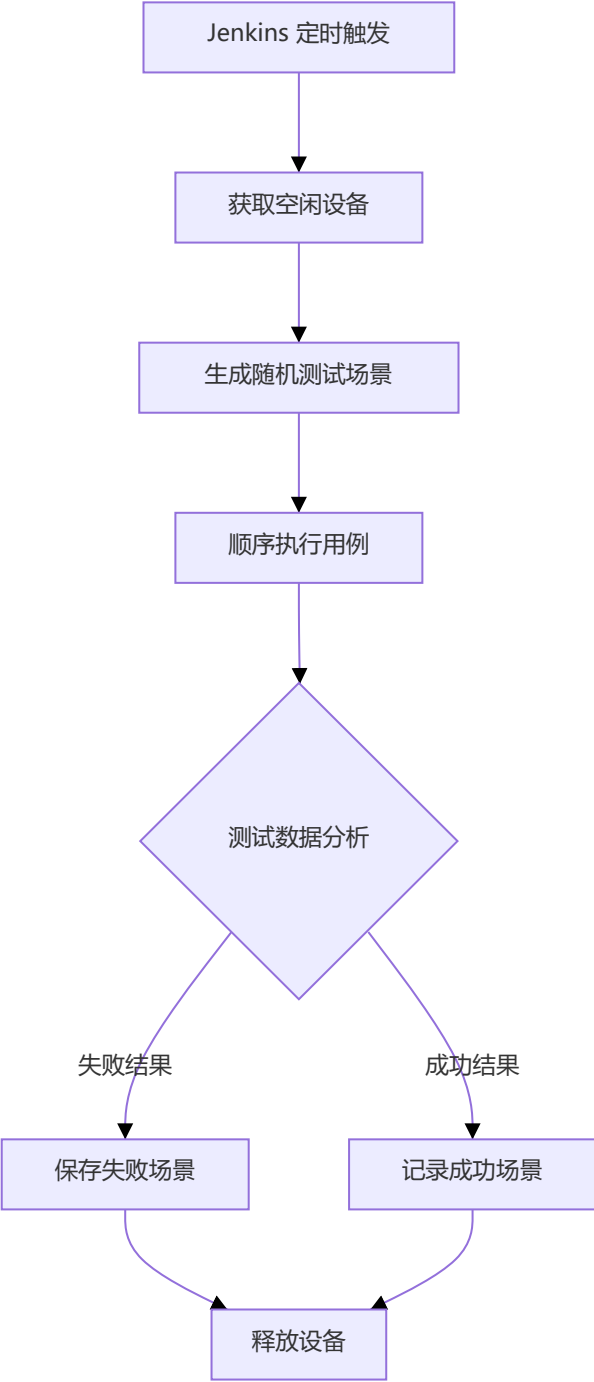
4.1 单用例执行与失败重试流程



对应数据说明

数据类型	说明
case_id	用例唯一标识
execution_id	单次执行唯一ID
retry_index	第几次重试
asr_result	识别结果
device_state_before	执行前设备状态快照
device_state_after	执行后设备状态快照
state_diff	状态变化字段
firmware_version	固件版本
algorithm_version	算法版本
logs	设备日志 / 工具日志

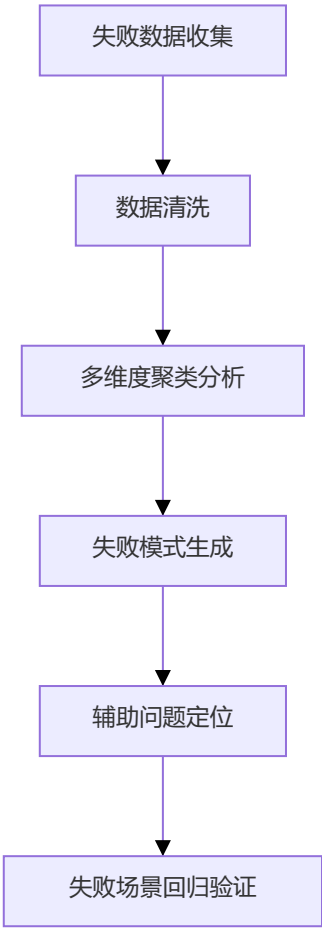
4.2 空闲设备随机场景执行流程



场景数据结构说明

字段	说明
scene_id	场景唯一标识
case_sequence	用例执行顺序
fail_case	失败用例
device_initial_state	场景初始设备状态
scene_events	状态变化事件
firmware_version	固件版本

4.3 失败分析与定位流程



分析维度说明

维度	用途
用例维度	识别高风险用例
固件 / 算法版本	判断回归或新引入问题
设备型号	硬件相关性分析
状态变化	网络 / 模式 / 音量等
场景序列	顺序依赖问题定位

五、模块化设计

模块	职责
用例执行模块	用例调度与结果判断
状态采集模块	设备状态快照
重试控制模块	用例级重试
场景生成模块	随机与规则组合
数据存储模块	结果与日志持久化
分析模块	失败聚类与统计

六、信息维度分析策略及数据标签

1.用例、场景维度

维度	权重/优先级
历史失败率	高
是否影响主链路	高
是否新改代码	中
是否状态敏感	中
是否偶现	高

根据不同权重生成待测试场景用例

2.失败用例「自动稳定性判定」

偶现、稳定复现、环境问题

3 次 retry 全失败 → STABLE

3 次 1 过 2 挂 → FLAKY

FAIL_TYPE:

- STABLE_FAIL （每次都失败）
- FLAKY_FAIL （偶现）
- ENV_RELATED （环境相关）

3.设备健康度标记

指标	示例
连续运行时长	12h
重启次数	3
平均 CPU	75%
失败率	20%
flash擦写次数	500

不同设备稳定性测试结果亦有差别，某些问题需要避开某些设备，避免测试因设备阻塞

4.日志特征指纹

error_code + key_stack + module，使用状态变更+当前设备状态组合的信息模式，可以快速判断新老问题、问题聚类。同时再不同版本验证时可以快速明确问题类型：新问题，回归问题，不影响固件的长期存在的低风险问题

5.用例状态标记

- 用例没恢复状态。前置用例环境重复设置| 首次设置
- 后续用例被污染

Case A 结束后：

必须满足 network=on, mode=default

否则：标记为“用例污染”等区别信息

6.失败用例、场景裁剪

结合前后状态变更、重试次数、用例标记等信息找出最小用例、场景节点

7.测试数据身份证

固件 v1.2.3

└─ 场景 SCENE_01
 └─ execution EXEC_123
 └─ failure FAIL_01

用于回归验证

7.总结

当前优化方案主要新增三个点：

1.数据采集

2.策略模块化（生成+分析两种类型策略）

3.结果分析